CSED233 · 데이터구조 · 포스텍

정리본 3. 알고리즘 분석

좋은 알고리즘의 조건 · 점근 분석 · Big-O/Ω/Θ · 최악/평균/최선.

1. 알고리즘과 성능

알고리즘(Algorithm): 문제를 유한한 시간 내에 해결하기 위한 단계별 절차.

예: 1부터 100까지의 합 — 입력 100 → 절차: sum에 1..100을 더함 → 출력 5050.

1.1 좋은 알고리즘의 5가지 조건

1. **입력(Input)**이 명확한가 — 배열, 정수, 문자열 등.
2. **출력(Output)**이 명확한가 — 정렬된 배열, 최댓값, true/false 등.
3. **명확성(Definiteness)** — "비슷한 것"이 아니라 "정확히 이것".
4. **유한성(Finiteness)** — 무한 루프 없음. `while(true)` ✗, `while(n--)` ✓.
5. **효율성(Effectiveness)** — 컴퓨터로 실제 실행 가능한 단계.

1.2 성능은 두 축으로 측정한다

- **Time complexity(시간복잡도)** $T(n)$: 입력 크기 n에 따라 몇 번의 연산? 예: 선형 탐색은 n번 비교 → $O(n)$.
- **Space complexity(공간복잡도)** $S(n)$: 입력 크기 n에 따라 얼마의 메모리? 예: 32비트 정수 배열은 4n bytes → $O(n)$.
- 실무 우선순위: ① 시간(사용자가 기다린다) ② 공간(메모리가 충분하면 덜 중요).

2. 두 가지 분석 방법

	Empirical(경험적 분석)	Theoretical(이론적 분석)
방법	실제로 구현 → 실행 → 시간 측정	고수준 설계를 수학적으로 계산

장점	실제 성능 데이터 · 환경(CPU/OS) 반영 · 직관적	환경 독립 · 일반적 예측 · 큰 n도 계산 가능
단점	구현 비용 · 환경 따라 결과 변동 · 큰 n 측정 곤란	실제와 괴리 가능 · 상수 인자 무시

```
// 경험적 분석의 예: 직접 재 본다
#include <ctime>
clock_t start = clock();
linearSearch(arr, target);
clock_t end = clock();
cout << "시간: " << (end - start) << " ms";
```

3. 점근 분석(Asymptotic Analysis): 왜 최고차항만 보는가

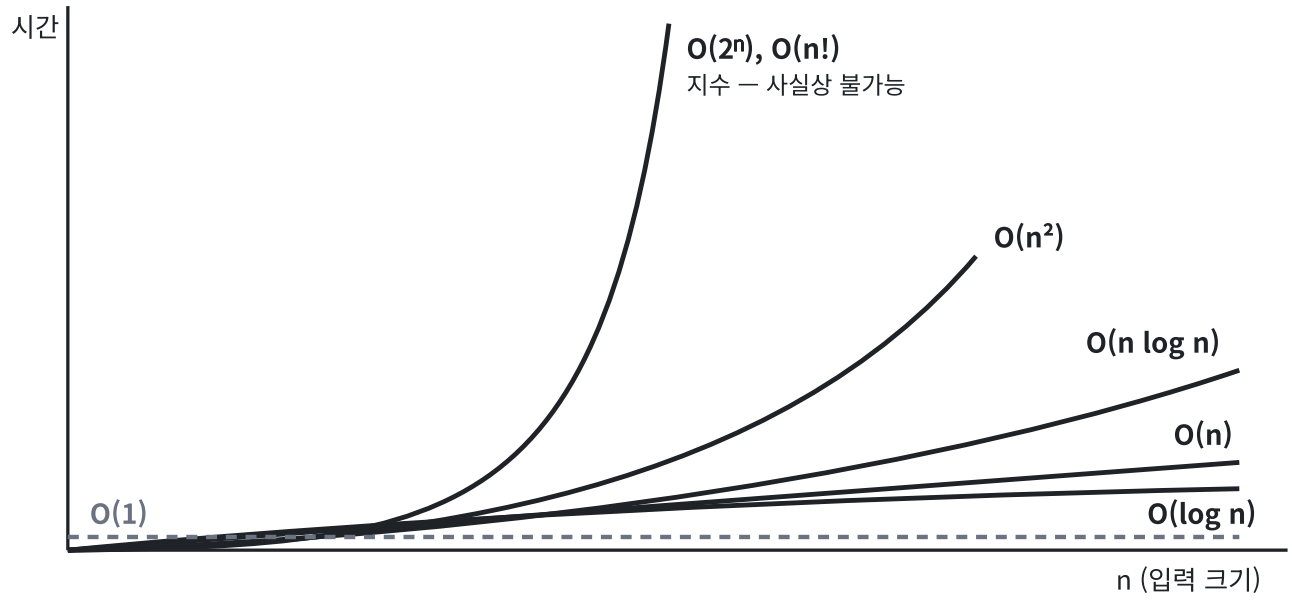
점근 분석은 n이 충분히 클 때의 극한 행동만 본다. 상수와 하위 항을 무시하고 지배 항만 남기는 이유를 수치로 확인해 보자.

$$T(n) = 2n^2 + 3n + 5$$

n	$2n^2$	3n	+5	T(n)	지배 항
1	2	3	5	10	상수항이 상대적으로 큼
10	200	30	5	235	n^2항이 지배적
100	20,000	300	5	20,305	n^2항이 거의 전부
1,000	2,000,000	3,000	5	≈ 2,000,000	오직 n^2만

→ n이 크면 **최고차항만 고려**: $O(n^2)$.

3.1 성장률 지형도



같은 n에서 위에 있을수록 느리다. 실무 표준은 O(n log n) 이하.

3.2 n = 1,000일 때 연산 횟수

각 복잡도 함수에 n = 1,000을 대입한 값 (실제 실행 횟수 아님).

시간복잡도	연산 횟수	체감
$O(1)$	1	극도로 빠름
$O(\log n)$	~10	매우 빠름
$O(n)$	1,000	빠름
$O(n \log n)$	10,000	중간
$O(n^2)$	1,000,000	느림
$O(n^3)$	1,000,000,000	매우 느림
$O(2^n)$	2^{1000}	불가능

4. 점근 표기법 3종

Big-O — 상한 ("최대 이 정도")

$$T(n) = O(f(n)) \iff \exists\, c > 0,\ n_0 \geq 1 \text{ s.t. } T(n) \leq c \cdot f(n)\ \ (\forall n \geq n_0)$$

- 알고리즘이 아무리 느려도 c·f(n)을 넘지 않는다는 **위쪽 경계**.

Big-Ω — 하한 ("최소 이 정도")

$$T(n) = \Omega(f(n)) \iff \exists\, c > 0,\ n_0 \geq 1 \text{ s. t. } T(n) \geq c \cdot f(n)\ \ (\forall n \geq n_0)$$

- 예: 배열을 전부 봐야 하는 작업은 최소 $\Omega(n)$.

Big-Θ — 동등 ("정확히 이 정도")

$$T(n) = \Theta(f(n)) \iff \exists\, c_1, c_2 > 0,\ n_0 \geq 1 :\ c_1 f(n) \leq T(n) \leq c_2 f(n)\ \ (\forall n \geq n_0)$$

- 예: Merge Sort는 최악·평균 모두 $\Theta(n \log n)$.

예제 · Big-O 증명 연습

T(n) = 2n + 3 이 O(n)임을 보이려면 2n + 3 ≤ c·n을 만족하는 상수 c를 찾으면 된다.

$$2n + 3 \leq 3n \iff n \geq 3 \Rightarrow 2n + 3 = O(n)$$

c = 3으로 잡으면 n ≥ 3에서 성립 → O(n) ✓. 같은 식으로 $3n^2 + 5n + 7 = O(n^2)$, $5 = O(1)$, $2^n = O(2^n)$.

5. 최악 / 평균 / 최선

배열 [3, 1, 4, 1, 5, 9, 2, 6]에서 선형 탐색을 예로 들면:

경우	의미	선형 탐색 예	중요도

Worst(최악)	가장 오래 걸리는 입력	6 검색(끝에 있음) 또는 7(없음) → 8회 → O(n)	**가장 중요** — 성능 보장의 기준
Average(평균)	모든 입력의 평균 (대상이 반드시 존재·각 위치 동일 확률 가정)	(1+2+⋯+8)/8 = 4.5회 → O(n) (실패 포함 시 입력 분포에 따라 달라짐)	현실적인 지표
Best(최선)	가장 빨리 끝나는 입력	3 검색 → 첫 원소에서 즉시 발견 → O(1)	거의 안 일어남, 지표 아님

실무에서는 이렇게 고른다

- **최악이 중요** — 실시간 시스템(의료, 항공), 금융 거래 → Merge Sort O(n log n) 보장.
- **평균이 중요** — 일반 데이터 처리, 온라인 게임 → Quick Sort 평균 O(n log n).
- **최선이 중요** — 거의 정렬된 데이터 → Insertion Sort (역전 쌍이 적을 때 효율적; 최선 O(n)).

6. Quiz

Q1. $O(n^2)$과 $O(n)$의 차이를 n = 1,000,000일 때 비교하라.

> 힌트

$n^2 = n \times n$. n에 10^6을 각각 대입해 보자.

> 정답

개념은 힌트 참고.

① $O(n)$: 10^6 = 100만 회 연산.

② $O(n^2)$: $(10^6)^2 = 10^{12}$ = 1조 회 연산.

③ 비율: $10^{12} / 10^6 = 10^6$.

정답: 100만 회 vs 1조 회 — 100만 배 차이.

Q2. O(n log n)은 어떤 의미인가?

힌트

$n \times \log_2(n)$. $\log_2(10^6) \approx 20$임을 이용해 $O(n^2)$과 비교해 보자.

정답

개념은 힌트 참고.

① $n = 10^6$이면 $\log_2(n) \approx 20$.

② $n \log_2(n) \approx 10^6 \times 20 = 2$천만 회.

③ $O(n^2) = 1$조 회와 비교하면 약 50,000배 빠름.

정답: 입력 n에 대해 대략 $n \times \log_2(n)$번의 연산. n = 100만이면 약 2천만 회로, $O(n^2)$보다 약 5만 배 빠르다.

Q3. 왜 최악의 경우(Worst Case)를 기준으로 분석하는가?

힌트

"보장"의 관점에서 생각해 보자. 평균은 입력 분포에 따라 달라지지만 최악은 어떤 입력이 와도 성립한다.

정답

개념은 힌트 참고.

① 평균 분석은 입력 분포 가정이 필요하고, 가정이 틀리면 무너진다.

② 최악 분석은 어떤 입력이든 "이 이상은 안 걸린다"를 보장한다.

③ 실시간·금융처럼 지연이 곧 사고인 시스템에서는 이 보장이 필수다.

정답: 최악의 경우 분석은 성능을 "보장"하기 때문 — 어떤 입력이 와도 걸리는 시간의 상한을 약속할 수 있다.

Q4. Bubble Sort $O(n^2)$와 Merge Sort $O(n \log n)$에서 n = 1,000일 때 차이는?

힌트

$\log_2(1000) \approx 10$. 각각 대입해 보자.

> **정답**

개념은 힌트 참고.

① Bubble Sort: $n^2 = 1000^2 = 100$만 회.

② Merge Sort: $n \log_2 n \approx 1000 \times 10 = 1$만 회.

③ 비율: 100만 / 1만 = 100배.

정답: 100만 회 vs 약 1만 회 — 100배 차이.

Q5. 경험적 분석과 이론적 분석의 차이는?

> **힌트**

측정이냐 계산이냐, 그리고 환경에 의존하느냐가 쟁점.

> **정답**

개념은 힌트 참고.

① 경험적: 실제로 구현해 실행 시간을 측정 → 현실적이지만 환경(CPU/OS)에 따라 달라진다.

② 이론적: 수학적으로 연산 횟수를 계산 → 환경 독립적이고 일반화되지만 상수는 무시한다.

정답: 경험적 = 구현 후 측정(환경 의존), 이론적 = 수학적 계산(환경 독립, 일반적).